

WORKSHOP

Not Every Agent Needs a Frontier Model

CYPHER 2026 | Nitin Agarwal

What we will cover

01

Stage classification

SLM-appropriate stages (classification, extraction, routing, reranking) vs. LLM stages (synthesis, ambiguity, judgment).

02

Hybrid pipeline design

Explicit per-stage model selection — not “call the biggest model for everything.”

03

Passive RAG → agentic

Query rewriting, multi-agent routing, optional self-reflection — and whether the extra reasoning is worth it.

04

Real metrics

Quality, latency (P50/P95), cost per query, SLM-vs-LLM call mix — not just a plausible-looking answer.

Small Language Models

The count heuristic

- Anything under roughly 10 billion parameters is treated as small, as of 2025.
- Frontier LLMs sit at hundreds of billions — GPT models are reported above a trillion.
- Easy to say, but it ages badly. Today's "large" is next year's phone model.

The deployability definition

- An SLM is a model that fits on a common consumer device and answers fast enough to serve one user's agentic requests.
- An LLM is simply everything that isn't.
- Anchored on what you can actually ship

LLM: The default is expensive

Most multi-agent systems route every call to the largest model — maximizing quality on hard tasks and wasting budget on easy ones.

Latency

Frontier models add seconds per call.
Classification and extraction rarely need that depth.

Cost

Token price scales with model size. Easy stages often dominate total call volume.

Capacity

GPU and rate limits are finite. Spending them on rewrite steps starves judgment stages.

Thesis: match model size to stage difficulty — then measure it.

Which stages need a frontier model?

SLM-APPROPRIATE

Small models are enough

Classification

Intent, validity, topic tags

Extraction

Keywords, entities, fields

Routing

Phase / agent handoff

Reranking

Lightweight relevance

Constrained rewrite

Short keyword queries

LLM-REQUIRED

Still need stronger models

Open-ended synthesis

Plans, long reports

Ambiguity resolution

Conflicting evidence

Multi-criteria judgment

AGI scoring (10 axes)

Self-reflection

Critique / gap analysis

Nuanced ranking

Subtle quality differences

Agent orchestration graph

Lead supervisor routes work; specialists report completion back

What this graph shows

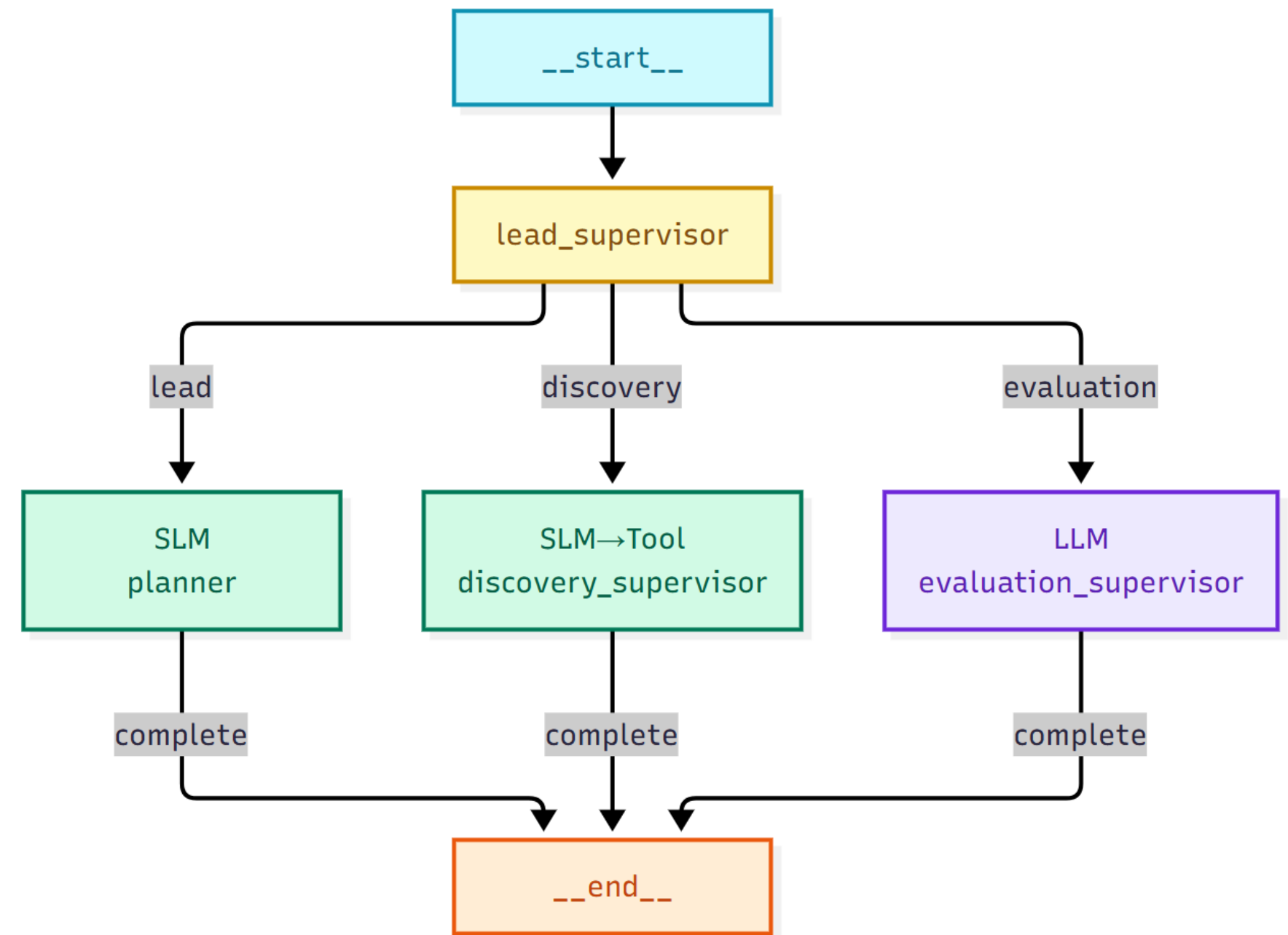
A lead supervisor dispatches to specialist agents (planner, discovery, evaluation). Each specialist can use a different model size. Completion flows back — not every node needs an 8B brain.

Hybrid policy on this graph

planner + discovery → SLM (0.5B)

evaluation + report → LLM (8B)

Routing stays explicit via `choose_model(stage, mode)`.



Design principles

1

Classify stages first

Label each step as classification / extraction / routing vs. judgment / synthesis before picking a model.

2

Default to SLM, escalate to LLM

Use the small model unless the stage requires open-ended reasoning or multi-criteria judgment.

3

Make routing explicit

A `choose_model(stage, mode)` policy beats “always call the biggest model.”

4

Measure the mix

Track quality, P50/P95 latency, cost, and SLM vs LLM call share — not just whether the answer looks good.

5

Agentic only when it pays

Query rewrite and multi-agent routing help when discovery is fragile; pure judgment stages still need the LLM.

Not every agent needs a frontier model.

Match model size to stage difficulty.
Measure quality, latency, cost, and call mix.